

WINDOW FUNCTIONS ASSIGNMENT

SQL Questions Q251–Q300

```
-- =====
-- SECTION 6 – WINDOW FUNCTIONS
-- =====

-- Q251. Assign a row number to all products based on price from highest to lowest.
SELECT product_id, product_name, price,
       ROW_NUMBER() OVER (ORDER BY price DESC) AS row_num
FROM products;

-- Q252. Assign a row number to all products based on price from lowest to highest.
SELECT product_id, product_name, price,
       ROW_NUMBER() OVER (ORDER BY price ASC) AS row_num
FROM products;

-- Q253. Rank all products based on price from highest to lowest.
SELECT product_id, product_name, price,
       RANK() OVER (ORDER BY price DESC) AS price_rank
FROM products;

-- Q254. Rank all products based on stock from highest to lowest.
SELECT product_id, product_name, stock,
       RANK() OVER (ORDER BY stock DESC) AS stock_rank
FROM products;

-- Q255. Apply dense rank to products based on price.
SELECT product_id, product_name, price,
       DENSE_RANK() OVER (ORDER BY price DESC) AS price_rank
FROM products;

-- Q256. Apply dense rank to products based on stock.
SELECT product_id, product_name, stock,
       DENSE_RANK() OVER (ORDER BY stock DESC) AS stock_rank
FROM products;

-- Q257. Assign row numbers to products within each category.
SELECT product_id, product_name, category_id, price,
       ROW_NUMBER() OVER (
         PARTITION BY category_id
         ORDER BY price DESC
       ) AS row_num
FROM products;

-- Q258. Rank products by price within each category.
SELECT product_id, product_name, category_id, price,
       RANK() OVER (
         PARTITION BY category_id
         ORDER BY price DESC
       ) AS price_rank
FROM products;

-- Q259. Dense rank products by price within each category.
SELECT product_id, product_name, category_id, price,
       DENSE_RANK() OVER (
         PARTITION BY category_id
         ORDER BY price DESC
       ) AS price_rank
FROM products;

-- Q260. Assign row numbers to products within each brand.
SELECT product_id, product_name, brand_id, price,
       ROW_NUMBER() OVER (
         PARTITION BY brand_id
         ORDER BY price DESC
       ) AS row_num
FROM products;

-- Q261. Rank products by price within each brand.
SELECT product_id, product_name, brand_id, price,
       RANK() OVER (
```

```

        PARTITION BY brand_id
        ORDER BY price DESC
    ) AS price_rank
FROM products;

-- Q262. Dense rank products by stock within each brand.
SELECT product_id, product_name, brand_id, stock,
       DENSE_RANK() OVER (
           PARTITION BY brand_id
           ORDER BY stock DESC
       ) AS stock_rank
FROM products;

-- Q263. Find the running total of product stock ordered by product ID.
SELECT product_id, product_name, stock,
       SUM(stock) OVER (
           ORDER BY product_id
       ) AS running_stock
FROM products;

-- Q264. Find the running average of product prices.
SELECT product_id, product_name, price,
       AVG(price) OVER (
           ORDER BY product_id
       ) AS running_average_price
FROM products;

-- Q265. Display each product price along with the average price of all products.
SELECT product_id, product_name, price,
       AVG(price) OVER () AS overall_average_price
FROM products;

-- Q266. Display each product stock along with the total stock of all products.
SELECT product_id, product_name, stock,
       SUM(stock) OVER () AS total_stock
FROM products;

-- Q267. Display each product along with the number of products in its category.
SELECT product_id, product_name, category_id,
       COUNT(*) OVER (
           PARTITION BY category_id
       ) AS category_product_count
FROM products;

-- Q268. Display each product along with the average price of its category.
SELECT product_id, product_name, category_id, price,
       AVG(price) OVER (
           PARTITION BY category_id
       ) AS category_average_price
FROM products;

-- Q269. Display each product along with the total stock of its category.
SELECT product_id, product_name, category_id, stock,
       SUM(stock) OVER (
           PARTITION BY category_id
       ) AS category_total_stock
FROM products;

-- Q270. Display each product along with the number of products in its brand.
SELECT product_id, product_name, brand_id,
       COUNT(*) OVER (
           PARTITION BY brand_id
       ) AS brand_product_count
FROM products;

-- Q271. Display each product along with the average price of its brand.
SELECT product_id, product_name, brand_id, price,
       AVG(price) OVER (
           PARTITION BY brand_id
       ) AS brand_average_price
FROM products;

-- Q272. Display each product along with the total stock of its brand.
SELECT product_id, product_name, brand_id, stock,

```

```

        SUM(stock) OVER (
            PARTITION BY brand_id
        ) AS brand_total_stock
FROM products;

```

```

-- Q273. Assign row numbers to orders based on total amount.
SELECT order_id, customer_id, total_amount,
       ROW_NUMBER() OVER (
           ORDER BY total_amount DESC
       ) AS row_num
FROM orders;

```

```

-- Q274. Rank orders based on total amount.
SELECT order_id, customer_id, total_amount,
       RANK() OVER (
           ORDER BY total_amount DESC
       ) AS amount_rank
FROM orders;

```

```

-- Q275. Dense rank orders based on total amount.
SELECT order_id, customer_id, total_amount,
       DENSE_RANK() OVER (
           ORDER BY total_amount DESC
       ) AS amount_rank
FROM orders;

```

```

-- Q276. Assign row numbers to orders for each customer.
SELECT order_id, customer_id, order_date, total_amount,
       ROW_NUMBER() OVER (
           PARTITION BY customer_id
           ORDER BY order_date
       ) AS order_number
FROM orders;

```

```

-- Q277. Rank orders by amount for each customer.
SELECT order_id, customer_id, total_amount,
       RANK() OVER (
           PARTITION BY customer_id
           ORDER BY total_amount DESC
       ) AS amount_rank
FROM orders;

```

```

-- Q278. Display each order along with the average order amount of that customer.
SELECT order_id, customer_id, total_amount,
       AVG(total_amount) OVER (
           PARTITION BY customer_id
       ) AS customer_average_order
FROM orders;

```

```

-- Q279. Display each order along with the total amount spent by that customer.
SELECT order_id, customer_id, total_amount,
       SUM(total_amount) OVER (
           PARTITION BY customer_id
       ) AS customer_total_spending
FROM orders;

```

```

-- Q280. Display each order along with the number of orders placed by that customer.
SELECT order_id, customer_id,
       COUNT(*) OVER (
           PARTITION BY customer_id
       ) AS customer_order_count
FROM orders;

```

```

-- Q281. Calculate running total of order amounts by order date.
SELECT order_id, order_date, total_amount,
       SUM(total_amount) OVER (
           ORDER BY order_date, order_id
       ) AS running_total
FROM orders;

```

```

-- Q282. Calculate running average of order amounts.
SELECT order_id, order_date, total_amount,
       AVG(total_amount) OVER (
           ORDER BY order_date, order_id
       ) AS running_average

```

```
FROM orders;
```

```
-- Q283. Display each order with the previous order amount using LAG.
SELECT order_id, order_date, total_amount,
       LAG(total_amount) OVER (
         ORDER BY order_date, order_id
       ) AS previous_order_amount
FROM orders;
```

```
-- Q284. Display each order with the next order amount using LEAD.
SELECT order_id, order_date, total_amount,
       LEAD(total_amount) OVER (
         ORDER BY order_date, order_id
       ) AS next_order_amount
FROM orders;
```

```
-- Q285. Display each order with the difference from the previous order amount.
SELECT order_id, order_date, total_amount,
       total_amount -
       LAG(total_amount) OVER (
         ORDER BY order_date, order_id
       ) AS amount_difference
FROM orders;
```

```
-- Q286. Display each order with the difference from the next order amount.
SELECT order_id, order_date, total_amount,
       LEAD(total_amount) OVER (
         ORDER BY order_date, order_id
       ) - total_amount AS next_amount_difference
FROM orders;
```

```
-- Q287. Calculate running total of order amounts for each customer.
SELECT order_id, customer_id, order_date, total_amount,
       SUM(total_amount) OVER (
         PARTITION BY customer_id
         ORDER BY order_date, order_id
       ) AS customer_running_total
FROM orders;
```

```
-- Q288. Display previous order amount for each customer.
SELECT order_id, customer_id, order_date, total_amount,
       LAG(total_amount) OVER (
         PARTITION BY customer_id
         ORDER BY order_date, order_id
       ) AS previous_customer_order
FROM orders;
```

```
-- Q289. Display next order amount for each customer.
SELECT order_id, customer_id, order_date, total_amount,
       LEAD(total_amount) OVER (
         PARTITION BY customer_id
         ORDER BY order_date, order_id
       ) AS next_customer_order
FROM orders;
```

```
-- Q290. Assign row numbers to order items based on price.
SELECT order_item_id, order_id, product_id, price,
       ROW_NUMBER() OVER (
         ORDER BY price DESC
       ) AS row_num
FROM order_items;
```

```
-- Q291. Rank order items based on price.
SELECT order_item_id, order_id, product_id, price,
       RANK() OVER (
         ORDER BY price DESC
       ) AS price_rank
FROM order_items;
```

```
-- Q292. Dense rank order items based on quantity.
SELECT order_item_id, order_id, product_id, quantity,
       DENSE_RANK() OVER (
         ORDER BY quantity DESC
       ) AS quantity_rank
FROM order_items;
```

```

-- Q293. Assign row numbers to order items for each product.
SELECT order_item_id, product_id, quantity, price,
       ROW_NUMBER() OVER (
         PARTITION BY product_id
         ORDER BY price DESC
       ) AS product_row_num
FROM order_items;

-- Q294. Calculate total quantity ordered for each product using a window function.
SELECT order_item_id, product_id, quantity,
       SUM(quantity) OVER (
         PARTITION BY product_id
       ) AS product_total_quantity
FROM order_items;

-- Q295. Calculate average order-item price for each product.
SELECT order_item_id, product_id, price,
       AVG(price) OVER (
         PARTITION BY product_id
       ) AS product_average_price
FROM order_items;

-- Q296. Calculate running quantity for each product.
SELECT order_item_id, product_id, quantity,
       SUM(quantity) OVER (
         PARTITION BY product_id
         ORDER BY order_item_id
       ) AS running_quantity
FROM order_items;

-- Q297. Display previous order-item price for each product using LAG.
SELECT order_item_id, product_id, price,
       LAG(price) OVER (
         PARTITION BY product_id
         ORDER BY order_item_id
       ) AS previous_price
FROM order_items;

-- Q298. Display next order-item price for each product using LEAD.
SELECT order_item_id, product_id, price,
       LEAD(price) OVER (
         PARTITION BY product_id
         ORDER BY order_item_id
       ) AS next_price
FROM order_items;

-- Q299. Display each order item along with total quantity of all order items.
SELECT order_item_id, order_id, product_id, quantity,
       SUM(quantity) OVER () AS total_quantity
FROM order_items;

-- Q300. Display order items with row number, rank and dense rank based on price.
SELECT order_item_id, order_id, product_id, price,
       ROW_NUMBER() OVER (
         ORDER BY price DESC
       ) AS row_num,
       RANK() OVER (
         ORDER BY price DESC
       ) AS price_rank,
       DENSE_RANK() OVER (
         ORDER BY price DESC
       ) AS dense_price_rank
FROM order_items;

```